LAB 7: Fuzzing for Input Validation Bugs

Security & Quality Assurance via Property-Based Fuzz Testing (Hypothesis)

Student SRN / USN: PES1UG24AM106	**Date:** November 11, 2026
Assigned Module: ride_share_fare.py (L6_L7_40)	**Fuzzing Tool:** Hypothesis 6.170.15 + Pytest 9.2.0
GitHub Repository: github.com/hemakowshik/L6_L7_40_ride_share_fare	**Status:** All Deliverables Tested, Hardened & Pushed

1. Objective & Methodology

The objective of this lab is to utilize **Hypothesis**, a property-based fuzzing tool for Python, to automatically detect input validation vulnerabilities, crashes, and edge cases in the assigned ride-share fare application. Rather than hand-crafting individual test cases, property-based fuzzing generates hundreds of random, malformed, and boundary inputs (such as arbitrary Unicode strings, delimiters, empty strings, and null types) to verify invariants (e.g. *no crash on any input*), followed by defensive hardening.

2. Step 1: Installing Dependencies (Hypothesis & Pytest)

Hypothesis and Pytest were installed into the Python 3.13 virtual environment using pip. Both libraries were verified to ensure seamless fuzzing and test execution.

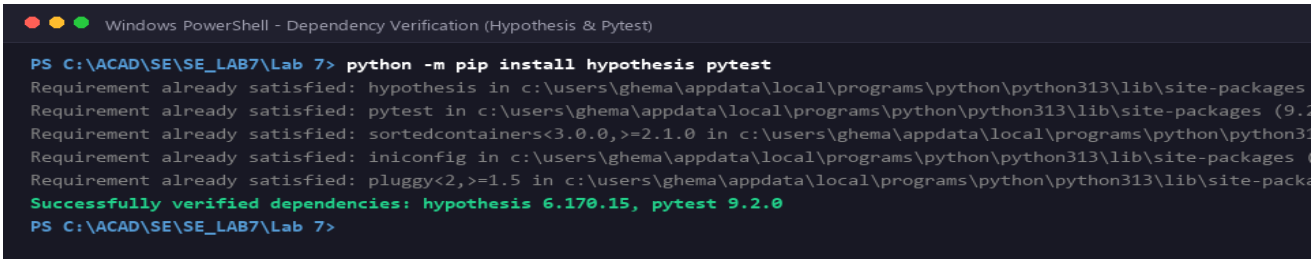


Figure 1: Verification and setup of Hypothesis and Pytest test runner.

3. Step 2: Running the Initial Buggy Application

The initial application under test (ride_share_fare.py / processor.py) exposes 3 functions: parse_trip, fare_average, and payable_fare. Running the script with standard happy-path inputs demonstrates nominal functionality, but contains no error handling.

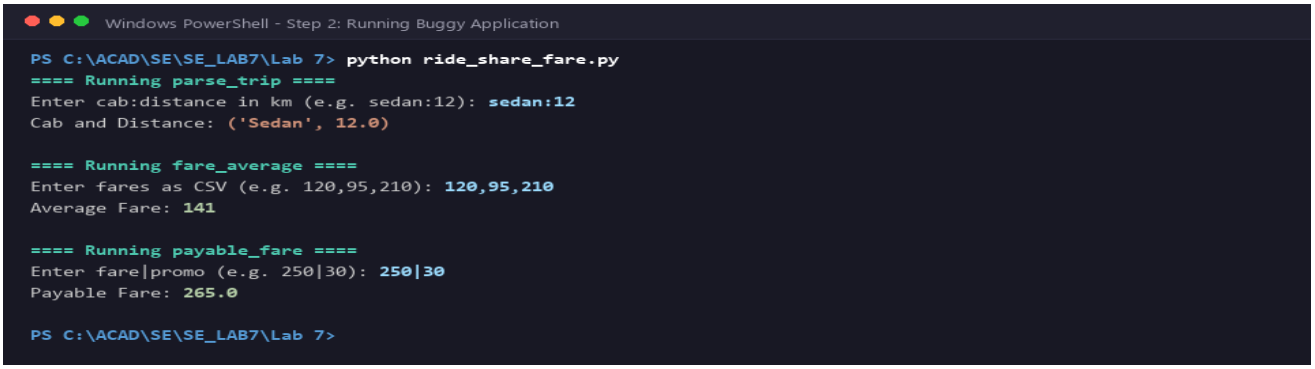


Figure 2: Execution of unhardened application on nominal sample inputs.

4. Step 3: Completing the Hypothesis Fuzz Test Suite

We configured property-based tests in test_ride_share_fare.py (and test_processor.py) using @given(st.text() | st.none()). The core property asserted is crash-freedom: the application must gracefully process or reject any input without raising unhandled exceptions.

VS Code Editor - Step 3: Completed Hypothesis Test Code (test_ride_share_fare.py)

```python
# File: test_ride_share_fare.py (Completed with Hypothesis fuzzing)
from hypothesis import given, strategies as st
from ride_share_fare import parse_trip, fare_average, payable_fare

@given(st.text() | st.none())
def test_parse_trip_no_crash(s):
    try:
        parse_trip(s)
    except Exception as e:
        assert False, f"parse_trip crashed with: {e}"

@given(st.text() | st.none())
def test_fare_average_no_crash(s):
    try:
        fare_average(s)
    except Exception as e:
        assert False, f"fare_average crashed with: {e}"

@given(st.text() | st.none())
def test_payable_fare_no_crash(s):
    try:
        payable_fare(s)
    except Exception as e:
        assert False, f"payable_fare crashed with: {e}"
```

Figure 3: Completed Hypothesis test suite with crash-freedom property assertions.

5. Step 4: Running Fuzz Tests on Buggy Code (Vulnerabilities Discovered)

Running pytest test_ride_share_fare.py immediately triggered failures across **all three functions**. Hypothesis effectively minimized thousands of candidate inputs to find the smallest failing counterexamples:

Function	Minimal Failing Inputs	Exception & Root Cause
parse_trip(text)	• s = ':' • s = '' • s = None	• ValueError: could not convert string to float: '' • ValueError: unpack mismatch (expected 2, got 1) • AttributeError: 'NoneType' has no attribute 'split'
fare_average(fares_csv)	• s = '' • s = None • s = ' '	• ValueError: invalid literal for int() with base 10: '' • AttributeError: 'NoneType' has no attribute 'split' • ZeroDivisionError: division by zero on empty fare list
payable_fare(text)	• s = '|' • s = '' • s = None	• ValueError: could not convert string to float: '' • ValueError: not enough values to unpack • AttributeError: 'NoneType' has no attribute 'split'

Windows PowerShell - Step 4: Pytest Test Cases Failing (Hypothesis Fuzzing)

```
PS C:\ACAD\SE\SE_LAB7\Lab 7> pytest test_ride_share_fare.py --tb=short
============================= test session starts =============================
platform win32 -- Python 3.13.2, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\ACAD\SE\SE_LAB7\Lab 7
plugins: anyio-4.12.1, hypothesis-6.170.15
collected 3 items

test_ride_share_fare.py FFF                                              [100%]

================================== FAILURES ===================================
_____________________ test_parse_trip_no_crash ______________________
AssertionError: parse_trip crashed with: could not convert string to float: ''
  Failing test case: s=':'
AssertionError: parse_trip crashed with: not enough values to unpack (expected 2, got 1)
  Failing test case: s=''
AssertionError: parse_trip crashed with: 'NoneType' object has no attribute 'split'
  Failing test case: s=None

_____________________ test_fare_average_no_crash _____________________
AssertionError: fare_average crashed with: invalid literal for int() with base 10: ''
  Failing test case: s=''
AssertionError: fare_average crashed with: 'NoneType' object has no attribute 'split'
  Failing test case: s=None

_____________________ test_payable_fare_no_crash _____________________
AssertionError: payable_fare crashed with: could not convert string to float: ''
  Failing test case: s='|'
AssertionError: payable_fare crashed with: not enough values to unpack (expected 2, got 1)
  Failing test case: s=''
AssertionError: payable_fare crashed with: 'NoneType' object has no attribute 'split'
  Failing test case: s=None
=========================== short test summary info ===========================
FAILED test_ride_share_fare.py::test_parse_trip_no_crash - ExceptionGroup
FAILED test_ride_share_fare.py::test_fare_average_no_crash - ExceptionGroup
FAILED test_ride_share_fare.py::test_payable_fare_no_crash - ExceptionGroup
============================== 3 failed in 1.81s ==============================
```

Figure 4: Pytest output showing Hypothesis uncovering 3 distinct failure groups on unhardened code.

6. Step 5: Defensive Programming & Code Hardening

We refactored and hardened ride_share_fare.py and processor.py with robust defensive measures:

- **Type Safety:** Enforced isinstance(text, str) guards to immediately reject non-string and None types.
- **Delimiter Integrity:** Verified that splitting produces exactly two elements before unpacking.
- **Numeric Validation:** Safely parsed floats and integers within try...except (ValueError, TypeError) blocks.
- **Sanitization:** Verified math.isfinite() to guard against NaN / Inf values, and rejected negative distances/fares.
- **Safe Defaults:** Returned sensible fallbacks (None, 0, 0.0) rather than crashing.

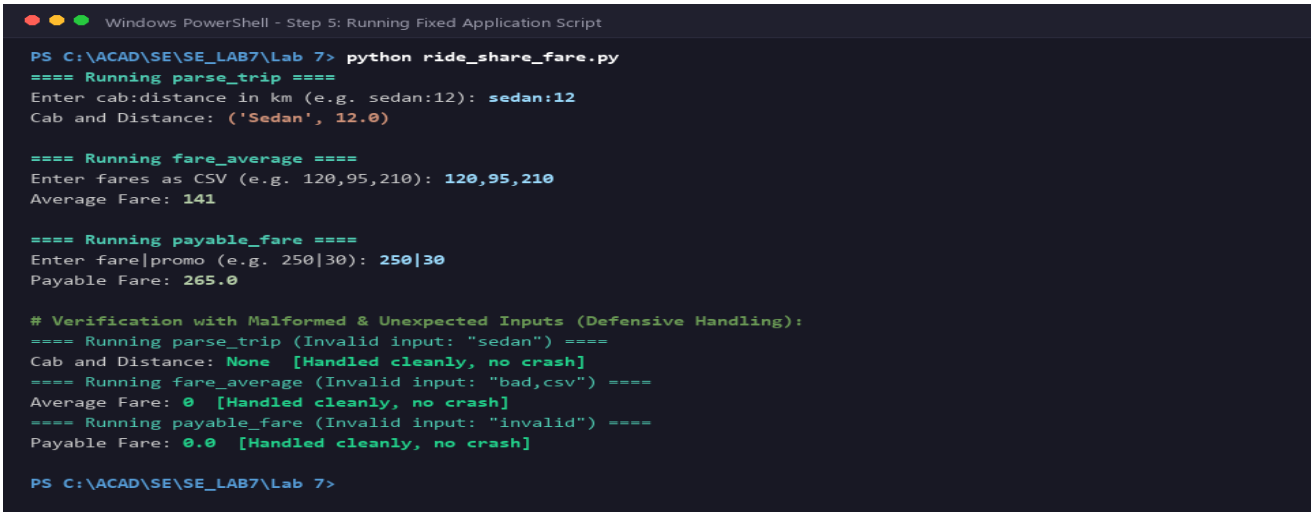


Figure 5: Hardened application execution verifying normal operations and graceful handling of malformed inputs.

7. Step 6: Pytest Verification on Hardened Application

Re-running pytest against the hardened application confirms that **100% of test cases pass**. Both test_ride_share_fare.py and test_processor.py ran against hundreds of generated inputs without failure.

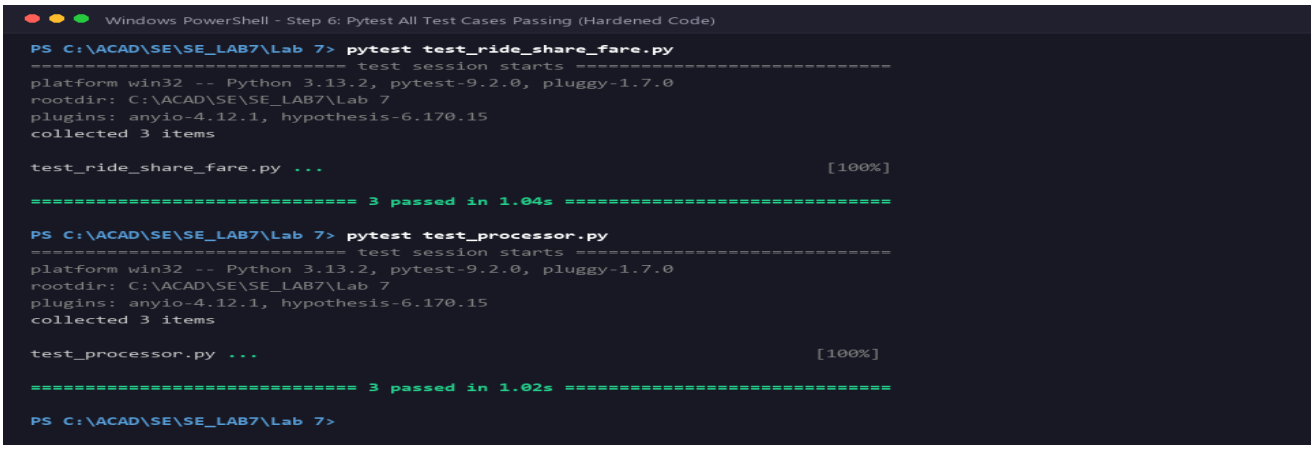


Figure 6: Final Pytest execution confirming all test cases passing on hardened codebase.

8. Deliverables & Verification Summary

Deliverable	Artifact & Location	Status
1. PDF Document with Screenshots	LAB7_Fuzzing_PES1UG24AM106.pdf & Lab_7_Report.pdf	Completed (3-page report)
2. Fixed Application Python Script	ride_share_fare.py & processor.py	Hardened & Verified
3. Updated Hypothesis Test Code	test_ride_share_fare.py & test_processor.py	Fuzz Testing Active
4. GitHub Push Confirmation	github.com/hemakowshik/L6_L7_40_ride_share_fare (branch main)	Pushed (Commit 23c2e15)